

Claude Science: the advanced guide

Extending and orchestrating the Research Toolkit on Claude Science — bundle v2.11.

This is the human-readable twin of `CS_ADVANCED.machine.md`, which is the authoritative source. Every fact here is a translation of an atom in that machine root; when the two disagree, the machine root wins and this file should be re-derived from it. Counts were re-computed on 2026-08-02 from the shipped `crt_science_bundle.json` (52 skills, 18 profiles) — never the Claude Code figures of “21 skills + 5 agents.” The content is folded and re-expressed from the Claude Code toolkit’s advanced chapters, translated into Claude Science’s own primitives; where a Claude Code idea has no Science counterpart it is named honestly rather than forced (see the section on what Claude Code has that Science lacks).

This guide is for a user who is already past the quickstart and the detailed usage guide, and who now wants two things: to get the most out of Claude Science, and to add their own capability to it. It splits into an architecture half (what the customizations are, and the API that drives them) and an orchestration half (how to wield them across delegated work, loops, and the memory that persists between sessions).

The one idea, and the map

The tying idea behind every tactic in this guide is that the harness beats the model. The model is an engine; the harness is the whole car around it. The fuel lines are context, the dashboard is the active profile’s system prompt together with the project memory that recalls automatically, and the attachments are the skills and their kernels. Tuning the car beats swapping the engine. Claude Science’s behavior is shaped by that layered instruction system — the profile persona, the skill descriptions the system keeps in view, the deterministic gates a kernel runs, the memory that returns each session — not by the raw weights. So the first move on any task is to pick the mechanism whose context cost and authority fit the job: an always-true directive belongs in a profile’s system prompt or an always-on skill; a procedure belongs in a skill loaded on demand; a deterministic check belongs in a kernel gate, not in a prose instruction that the model may or may not honor.

The reason the guide is organized the way it is comes down to one constraint: context is a scarce, depleting budget. As the number of tokens in the window grows, the model’s ability to accurately recall any given item degrades — a transformer’s pairwise attention gets stretched thin. This is context rot, and treating context as a depleting attention budget is what every method here ultimately serves. Targeted inclusion, session hygiene, and the isolation of delegated children are all context-preservation moves, not stylistic preferences (Effective context engineering).

Two corollaries follow. First, include on target rather than on spec: aim for the smallest set of high-signal tokens that maximizes the odds of the outcome you want, the Goldilocks zone between too much (which dilutes attention) and too little (which leaves the model navigating blind). Name the in-scope objects up front, keep the auto-recalled memory dense, and re-scope a polluted context rather than fighting it. Second, search live rather than pre-indexing: retrieve at runtime — read

a workspace file, read a skill's body with `host.skills.read`, search the artifact store when a step needs it — instead of pre-loading everything or trusting a stale index. Hold the lightweight identifier (an artifact id, a skill name, a path) and open the object on demand.

The master constraint, then, is the finite and degrading context window. Every extension point and every workflow is a lever on what fills it and when, so the single question to judge any tactic by is whether it spends high-signal tokens or wastes the budget.

The rest of this guide covers, in order: the Science extension architecture of profiles, skills, and kernels; the `host.*` API surface that drives them; how to author skills well; profiles and delegation; orchestration and its patterns; memory as a poison surface, together with context engineering; authoring your own extensions to the bundle contract; an honest accounting of what Claude Code has that Science lacks; and a closing reference and glossary. One re-pointing note before the depth: Claude Code's three install scopes, its `~/ .claude` file-drop model, and its documentation links have no Science analogue. The Science structure is a single source tree, `bundle_src/{skills,profiles}`, built and installed into one account.

The Science extension architecture

Everything that customizes Claude Science is either a skill or a profile shipped in the bundle, and kernels are the deterministic helpers that skills carry. There are two shapes and one sidecar: a skill is a folder of instructions the session loads on demand; a profile is a named specialist you dispatch; a kernel is a `kernel.py` of gate and helper functions that a skill's prose calls instead of re-reasoning them each turn.

A skill is a folder, not a file. It lives at `bundle_src/skills/<name>/SKILL.md`, with frontmatter carrying a name and a description above the body, and it may include a `kernel.py`, a `references/` directory, and fixtures alongside. The system keeps every skill's description in view, while the body loads only when the skill is invoked — the progressive-disclosure mechanism covered below. Fifty-two skills ship in v2.11, and nineteen of them carry a kernel.

A profile is a `.json` specialist at `bundle_src/profiles/<NAME>.json`, defining a name, a `display_name`, a description, a `system_prompt`, and an optional `skillNames` list. Where a skill is a capability, a profile is a whole specialist persona that you dispatch through delegation. Eighteen profiles ship.

A kernel is a `kernel.py` sidecar: the deterministic functions — gates, routing tables, helpers — that a skill's instructions load and run rather than reconstructing every turn. You load one by executing the content the API hands back, `exec(host.skills.read("<skill>", "kernel.py")["content"])`, and a gate returns a verdict dictionary carrying a marker that reads PASS or FAIL. This is the Science expression of “ship scripts, don't re-derive boilerplate.”

Those source objects become a live account through a fixed pipeline. The source tree `bundle_src/{skills,profiles}` is compiled by `build_crt_science_bundle.py --src bundle_src --config build_config.json --out crt_science_bundle.json`, run from the bundle directory, and `install_crt_science.py` installs the result into the Science account. One architectural fact governs the whole bundle: the build discovers only the `skills/` and `profiles/` directories, so

the built JSON carries a `skills[]` array and a `profiles[]` array and nothing else. There is no document carrier in the bundle — which is exactly why this guide is a repo-side reference rather than an installed object.

The invariant to hold is that a customization’s kind is decided by its shape — a folder is a skill, a `.json` is a profile, a `kernel.py` is a sidecar — and the bundle installs into the one Science account. There is no per-scope reach to choose and no file-drop location that changes who receives a customization. The `host.*` calls that read and dispatch these objects come next; authoring one to the contract and rebuild gates is covered later; and the Claude Code scope, settings, and hooks surface this replaces is named in the honest-omissions section.

The `host.*` API surface

The action surface a Science session drives is the set of `host.*` calls, issued from a repl cell — the Python cell is the Science analog of Claude Code’s tool loop, and it is where you execute a kernel and call the API.

The `host.skills` calls read and edit skill files. `host.skills.read(name, file)["content"]` returns a skill’s `SKILL.md` or its `kernel.py`, and executing the kernel content loads its functions. `host.skills.edit(name, file, content)` edits a skill file and re-runs the sidecar gate in the same motion; a bad edit is refused with the gate’s message, and the discipline is never to retry-edit past a gate.

The `host.delegate` call is the heart of orchestration. `host.delegate([{"task": ..., "name": ..., "model": ... }])` dispatches one or more children, each running in its own context window, and returns each child’s result together with a model field and a frames record. It is the Science analog of Claude Code’s Task tool and its subagents.

The `host.mcp` call reaches a connector on the user’s own machine — for example the hands-free-alert audible beep that signals a turn boundary. Because those connectors run behind tunnels that churn, the call is fire-and-forget: wrap it in `try/except Exception: pass`, and never let a failed connector delay a turn or get narrated.

Two durable stores complete the surface, and the difference between them is load-bearing. Project memory auto-recalls across sessions — this is the poison surface discussed below, because only memory recalls automatically, so a stale row is re-read as current at every login. Artifacts and lineage, by contrast, are inert until searched: they do not auto-recall. `host.lineage[vid]["code"]` reproduces a shipped artifact’s own code, which makes it the primary record for any “what does it do now” question. The `host.artifacts` interface exposes versions, but `latest` resolves last-writer-wins, so pin an explicit `version_id` rather than trusting `latest`.

The invariant is that `host.*` is the whole action surface, and permission is the account’s rather than a per-command allow-or-deny list you tune. An unattended run is bounded by which connectors and stores exist, not by a settings deny-list — a sharp contrast with Claude Code. One capability is deliberately held back: a Science-native treatment of authoring your own `host.*` custom tool is deferred, and the honest-omissions section says so plainly.

Skills, mastered

Skills are the deepest extension point, because a skill turns a generalist into a specialist at your task, cheaply, through a folder the session loads only when it is relevant. The right way to picture building one is writing an onboarding guide for a new hire: the document a competent generalist needs to do this job, this team’s way (Equipping agents with Agent Skills).

Because a skill is a folder and not a file, it can carry a `kernel.py`, a set of references/, and fixtures beside `SKILL.md`, all discovered and loaded on demand. Progressive disclosure is the filesystem used as context engineering, and it works in three load levels: the description loads at session start and stays in view; the `SKILL.md` body loads only when the skill is judged relevant; and the references/* files load only as a task branch reaches for them. The context a skill can bundle is effectively unbounded, because you pay only for the branch a task actually touches (Equipping agents with Agent Skills). The description is capped at 1024 folded characters by the sidecar contract.

The description is load-bearing, and this is the one invariant of skill authoring: it is the only part loaded until a trigger fires, so it alone decides whether the skill ever activates. Write it as a specific, positively phrased, trigger-worded sentence that names when it should fire — including the words a user will actually say — rather than a vague summary. A precise description outweighs a perfect body the model never reaches.

A good skill fits exactly one of nine categories cleanly: a library or API reference, product verification, data fetching and analysis, business-process automation, code scaffolding, code quality and review, CI/CD and deployment, runbooks, or infrastructure ops.

A handful of authoring moves carry most of the leverage. Lead with the gotchas — the highest-signal section — and encode the traps you actually hit rather than the happy path the model already handles. Ship a kernel instead of making the model rebuild boilerplate: deterministic operations such as sorting, parsing, or a gate belong in `kernel.py`, which is cheaper and exact, loaded and run in a repl cell, so that turns are spent on composition rather than reconstruction; this is the Science form of “ship scripts.” Compose skills by name, referencing another installed skill inside your instructions, so that small single-purpose skills chain. Give latitude rather than rails, providing information and the freedom to adapt instead of a rigid step list that snaps when the situation differs. And skip the obvious, spending context on your conventions, your gotchas, and your non-obvious constraints rather than on generic practice the model already knows.

Profiles and delegation

Delegation hands a bounded job to a separate context window so that the main thread stays clean. A profile is a named specialist, and “delegation” names the relationship — the main thread calls it — rather than a different kind of thing. It is the Science analog of a Claude Code subagent: the same mechanism, expressed in Science’s primitives.

A profile dispatched through `host.delegate` runs in its own fresh context window, and only its distilled result returns to the main thread. Profiles are defined in `bundle_src/profiles/<NAME>.json` with a `name`, a `display_name`, a description that provides the auto-match text, a

system_prompt that sets the persona, and an optional skillNames list of skills it should load. Eighteen profiles ship, and GENERALIST is the catch-all.

To dispatch, call `host.delegate([{"task", "name", "model" }, ...]);` a single call can fan out to several children at once. Choose each child's model by task difficulty using the delegation-planning kernel's tier table, and never use `claude-opus-5` or the bare alias `opus` — that is a hard ban, and the `model_route_gate` kernel returns a FAIL verdict on it.

The invariant that makes delegation worth its cost is context isolation: a delegated child spends a separate context window, so its intermediate exploration and verification never accrue to the main thread — only the distilled summary returns. That isolation is the reason to delegate. Many children orchestrated together become a harness, which the next section covers; the same isolation is what keeps the main thread clean; and profiles are authored to the same contract and rebuild gates as skills.

Orchestration and its patterns

Orchestration is the set of repeatable ways to compose one model, its tools, and its children into production work, and the right one is picked by the shape of the task. The spine running through all of them is to give the child a check it can run. With no verifiable check, you are the verification loop, and every mistake waits for you to notice it. Give a pass/fail signal — a test suite, a gate's exit code, a kernel's PASS/FAIL marker, a fixture diff, a screenshot — and the loop closes on its own. Every pattern below is a way to manufacture that check.

Underneath sits one distinction. A workflow orchestrates models and tools through predefined code paths, where you own the control flow; an agent dynamically directs its own process at runtime, where the model owns it (Building effective agents). Most production value lives in workflows; reach for a full agent only when the path cannot be pre-drawn.

Five building blocks compose nearly everything, and the craft is to combine the fewest that solve the task. Prompt-chaining runs fixed sequential steps, each consuming the last one's output, with an optional gate between, and fits when the task cleanly splits into fixed subtasks. Routing classifies the input and dispatches to a specialist handler, and fits when distinct categories are better handled separately. Parallelization runs calls concurrently and aggregates them, in two forms — sectioning independent subtasks to run at once, and voting the same task N times for confidence. Orchestrator-workers has a lead dynamically split the task, delegate to children, and synthesize their results, and fits when you cannot predict the subtasks up front. Evaluator-optimizer has one call generate while a second critiques against criteria, looping until it passes, and fits when you have clear evaluation criteria and iteration measurably helps.

Several concrete workflows recur. Explore, plan, code, then commit separates research from execution so the model does not solve the wrong problem — the `plan` skill supports this. Test-first hands the child the pass/fail check the spine demands. Writer-and-reviewer exploits the fact that a fresh child cannot inherit the producer's bias: dispatch `CODE_REVIEW_DEBUGGER` as the adversarial reviewer, and tell it to flag only correctness and requirement gaps, because a gap-seeking reviewer left unconstrained drives over-engineering. And independent workstreams should be parallelized rather than run one at a time.

Loops are cycles that repeat until a stop condition, and there are four types, distinguished by what you hand off. A turn-based loop hands off the check, encoding your manual verification as a skill so the child self-verifies end to end; it fits short, one-off work. A goal-based loop hands off the stop condition — a deterministic, machine-checkable bar — which an evaluator re-checks each time the child tries to stop. A time-based loop hands off the trigger, an interval or an event, and fits recurring work and external systems. A proactive loop hands off the prompt, a standing and well-defined stream with no human in the loop. On Claude Science these four map to the four `host.delegate` topologies named by the delegation-planning skill: `parallel-wave`, `sequential-build`, `convergence`, and `verify-loop`. The Claude Code `/loop`, `/goal`, and `/schedule` slash commands themselves are Code-only.

When a harness assembles orchestrator-workers on the fly for a task, six patterns describe the shapes it takes: `classify-and-act`, `fan-out-and-synthesize`, `adversarial-verification` (a separate verifier judged against a rubric), `generate-and-filter`, `tournament` (where pairwise judgment beats absolute scoring), and `loop-until-done` (for work of unknown size).

The reason to spend separate children at all is that a single context accumulates three failure modes, and isolated children fix each. Agentic laziness is stopping before finishing a multi-part task. Self-preferential bias is preferring one's own results, especially when asked to verify them. Goal drift is the decay of fidelity to the objective across many turns, worsened by lossy compaction. The fix, in one line, is to give each job a separate child with a clean context.

Scaling multi-child work well rests on a few lessons (Multi-agent research system). Delegate explicitly: each child needs an objective, an output format, tool and source guidance, and clear boundaries, because vague briefs make children duplicate work and leave gaps. Scale effort to complexity: a simple fact-find is one child, a comparison is two to four, and hard research is ten or more with divided responsibilities. Minimize the game of telephone by having children write their outputs to artifacts and workspace files and pass lightweight references back, rather than routing everything through the coordinator's context — this is the supervisory-workflow discipline. And weigh the economics: agents use roughly four times the tokens of chat and multi-agent runs roughly fifteen times, while token usage alone explains about eighty percent of performance variance, so a harness is justified only when the task's value pays for the added spend.

Two closing disciplines keep the output trustworthy. Fix the system, not the instance: when a result misses the standard, encode the fix into the skill, kernel, or profile so that every future child clears it. And keep it simple: find the simplest thing that works, add machinery only when it demonstrably improves outcomes, never wrap a one-shot task in a loop, and pilot before a large fan-out, because a harness can spawn many children. The invariant across all of it is that the leverage is isolated contexts — a fresh verifier cannot inherit the producer's bias, and a per-task child cannot inherit the orchestrator's drift — and that separation is precisely what defeats laziness, self-preference, and goal drift.

Memory as a poison surface, and context engineering

Two things meet in this section: the master discipline of curating the smallest high-signal token set, and the Science-specific hazard that durable memory is what recalls automatically, so a stale memory row poisons every future session.

Context engineering is the successor to prompt engineering. Prompt engineering wrote the words of one turn; context engineering curates the whole token set the model sees each turn — the system prompt, the skills, the retrieved data, the history, and the memory — across a multi-turn loop (Effective context engineering). Small wins because attention is a finite budget that degrades as the window grows, so more tokens mean lower signal per token and diminishing returns; the objective is always the smallest high-signal set, never the largest context.

Four levers implement the discipline. Altitude tunes the system prompt to the Goldilocks zone — too low is brittle if-else logic that overfits, too high is vague guidance that gives no steer. Just-in-time loading holds lightweight identifiers, such as artifact ids, skill names, and paths, in the window and loads the underlying data at runtime. Compaction, as history nears the limit, summarizes and reinitializes a fresh window, preserving the load-bearing atoms — decisions, unresolved bugs, the contract — and dropping the redundant. Structured note-taking persists durable state outside the window and pulls it back only when relevant.

Here is where Claude Science differs sharply from Claude Code. On Science, durable memory recalls automatically across sessions, while artifacts are inert until searched. A durable state memory row frozen as “done,” “green,” or “51 skills” is therefore re-read as current at every login, silently misleading the session — that is passive staleness — and an injection that lands in a durable memory surface persists and is re-read as current, which is the adversarial version of the same problem. The currency discipline that answers it has four parts.

Keep one canonical memory row per topic and supersede it in place. Never append a second “current” row that disagrees with the first, because two current rows that diverge are the worst poison of all: the divergence is undetectable to a future reader. Keep done-records, snapshots, and closed logs as inert artifacts rather than memory rows, so they do not recall as if they were live. Favor provenance over description: answer a “what does it do now” question from the primary record — the shipped skill or profile bytes, or `host.lineage[vid][“code”]` — not from a memory row that describes it, and remember that agreement among several memory rows is zero additional evidence until one is confirmed to be the record, because non-independent descriptions echo rather than corroborate. Finally, verify the verifier: a count or status written into memory is a claim about a file and goes stale silently — “N skills” is true until someone adds the next one, and the sentence does not update itself — so never hardcode such a number. Re-derive it at check time and compare against what you just measured, using the `check_currency`, `provenance-guard`, and `verification-loop` gates, and never trust a gate you have not shown catches its target defect.

The invariant is that on Science the durable-memory surface is the poison surface, so the working rules are to keep one canonical row per topic, supersede in place, hold inert records as artifacts rather than memory, and re-derive every count and status at read time from the primary record.

Authoring your own extensions on Science

Building your own skill, profile, or kernel to the bundle's standard follows a repeatable loop. The document loop is to draft with the `machine-md` skill, which applies LLM-document best practices — positive trigger-conditioned framing, output-detectable triggers, atom-preservation — then audit through the `LLM_DOC_ARCHITECT` profile, since Claude Science ships no separate machine-doc-reviewer profile and the audit role folds into that profile, then render the human and PDF twin with `folio-science` (offline pandoc and typst) and the `doc-pipeline` skill. The machine root stays the authoritative source, and the human `.md` and PDF are derived from it.

To author a new skill, create `bundle_src/skills/<name>/SKILL.md` with a name and a trigger-phrased description of at most 1024 folded characters in the frontmatter, plus an optional `kernel.py`. The sidecar contract — the constraint that once broke an install, on 2026-07-28, and so must be honored from the start — governs any kernel: the module's top level holds plain-name function definitions, imports, and literal-constant assignments only. No computed values such as `re.compile`, `frozenset`, or any call; no underscore-prefixed names, whether definitions or assignments; and no top-level `if`, including the `__main__` guard. `check_sidecar_contract.py` must exit 0 before any build.

To author a new profile, create `bundle_src/profiles/<NAME>.json` with a name, a `display_name`, a trigger-phrased description, a `system_prompt` that sets the persona, and an optional `skillNames` list.

Rebuilding and gating follows the mirror discipline. Any edit under `bundle_src/` calls for a rebuild via `build_crt_science_bundle.py --src bundle_src --config build_config.json --out crt_science_bundle.json`, run from the bundle directory, followed by `check_bundle_parity.py` in its build, strict-bytes, and manifest modes, and then `check_sidecar_contract.py`. Never hand-edit `crt_science_bundle.json`. Then install with `install_crt_science(overwrite=True)`, because differing profiles are silently skipped without the overwrite. The toolkit-extension-authoring skill carries this contract. A TIER-C content change owes its Claude Code twin a re-expression in that platform's own primitives — `host.delegate` maps to the Task tool, artifacts map to files — never a byte-copy across the boundary. (TIER-C is the assignment in the twin-architecture tier model; TIER-S items share content and carrier alike, so a change there propagates as the same change.)

The invariant is that the machine root and source object are authoritative, while the human twin, the PDF, and the built JSON are derived, so you change behavior by editing the source skill or profile and rebuilding, never by editing the built JSON or a description of it. One last discipline governs claims of success: a new fix ships as attempted-untested until a measurement — the countermeasure-audit sweep — shows the failure rate has dropped. Existence is not efficacy, and there is no “works” without a cited measurement.

What Claude Code has that Science lacks

Naming the omissions keeps the port honest. The following Claude Code chapters have no Science analogue and are deliberately left out above.

Install scopes, the file-drop model, settings, and permissions have no equivalent. Claude Code's three scopes (user, project, managed), its `~/ .claude` file-drop, its `settings.json` precedence and deep-merge, and its permissions allow/ask/deny list are all absent on Science, which installs into one account where permission is the account's rather than a per-command deny-list you tune.

Hooks do not exist. Claude Code's deterministic event scripts — firing on tool use, prompt submission, stop, and roughly thirty other events, communicating over stdin JSON — have no counterpart, and there is no turn-end or event hook at all. Anything Claude Code does with a hook, Science must do agent-initiated; the turn-end audible beep, for instance, is the audible-alert skill fired by the profile itself, not a stop hook.

The stock slash-command catalog does not carry over, because those built-in verbs are Code CLI features; Science surfaces only the few authored as skills, namely `solo`, `plan`, and `collab`. The `/loop`, `/goal`, and `/schedule` loop-command mechanics likewise stay behind: the loop types fold into the orchestration section as `host.delegate` topologies, but the slash-command primitives are Code-only. MCP servers, plugins and marketplaces, headless `claude -p` and the Agent SDK, and OS-level sandboxing are all Code-platform features with no Science analogue — `host.mcp` reaches a single user-side connector, but it is not the MCP client/server ecosystem, not plugin bundling, and not a sandbox wall. And the Code-specific documentation links under `code.claude.com/docs/en/` are Code-only and are dropped from the reference list.

One genuine Science analogue is deferred rather than dropped. The transferable idea from Claude Code's outward-extension chapter — that the platform exposes tools you design for agents — maps to Science's `host.*` and custom-tool surface. A Science-native treatment of authoring your own `host.*` tool is held for a later wave, drawing on Writing tools for agents as its source principle, and it will be written natively rather than ported byte-for-byte from the Code MCP chapter.

References and glossary

The sources below are the platform-agnostic writings the folded concepts genuinely rest on; the Claude Code product documentation links are dropped as Code-only.

- **Building effective agents** — <https://www.anthropic.com/engineering/building-effective-agents> — the workflows-versus-agents distinction, the five building blocks, and the simplicity thesis.
- **Effective context engineering for AI agents** — <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents> — context as the master finite resource, context rot, just-in-time loading, compaction, and altitude.
- **Equipping agents for the real world with Agent Skills** — <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills> — skills as progressive-disclosure capability packages that cost context only on demand.
- **How we built our multi-agent research system** — <https://www.anthropic.com/engineering/multi-agent-research-system> — orchestrator-workers at scale, with the economics and scaling lessons.

- **Writing effective tools for AI agents** — <https://www.anthropic.com/engineering/writing-tools-for-agents> — tool-design principles, and the source for the deferred `host.*` authoring treatment.

The glossary, re-termed to Science:

- **profile** — a named specialist persona (`bundle_src/profiles/<NAME>.json`) dispatched via `host.delegate`; the Science analog of a Claude Code subagent.
- **skill** — a folder (`SKILL.md` plus an optional `kernel.py` and `references/`) whose description auto-loads and whose body loads on demand.
- **kernel (sidecar)** — a skill's `kernel.py` of deterministic gate and helper functions, loaded via `exec(host.skills.read(...)[ "content" ])`; a gate returns a marker verdict.
- **sidecar contract** — the top-level `kernel.py` rule (plain definitions, imports, and literal assignments only; no computed values, no underscore-prefixed names, no top-level `if` or `__main__`); `check_sidecar_contract.py` must exit 0 before a build.
- **host.delegate** — the API call that dispatches a profile into its own context window and returns its result; the Science analog of the Claude Code Task tool.
- **host.skills** — the API to read and edit a skill's files; `.edit` re-runs the sidecar gate.
- **project-memory** — durable memory that auto-recalls across sessions; the Science poison surface.
- **artifact and lineage** — durable stored objects, inert until searched; `host.lineage[vid][ "code" ]` reproduces the shipped bytes, the primary record; pin an explicit `version_id`, not latest.
- **progressive disclosure** — a skill loads only its description up front, with the body and references loading on demand, so you pay context only on demand.
- **context rot** — as the token count in the window grows, the model's accurate recall of any given item degrades, which is the reason to budget context.
- **context engineering** — curating the smallest high-signal token set the model sees each turn; the successor to prompt engineering.
- **the four topologies** — parallel-wave, sequential-build, convergence, and verify-loop, the `host.delegate` shapes the loop types map to.
- **the six patterns** — classify-and-act, fan-out-and-synthesize, adversarial-verification, generate-and-filter, tournament, and loop-until-done.
- **the five blocks** — prompt-chaining, routing, parallelization, orchestrator-workers, and evaluator-optimizer.
- **VLOOP (verify the verifier)** — re-derive a count or status at check time, and never trust a gate not shown to catch its defect.

The authoritative record for what the bundle currently is remains the shipped `crt_science_bundle.json`, read directly, not this prose — provenance over description.